UNIVERSITÀ degli STUDI di CATANIA

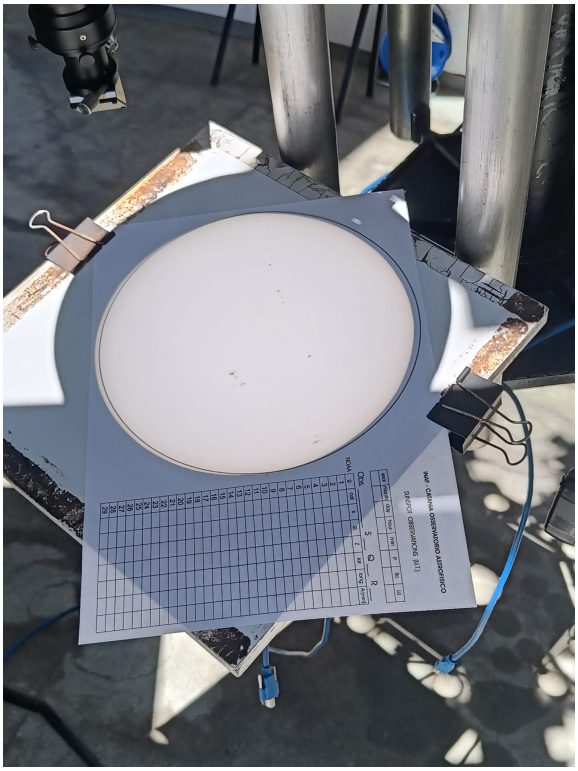
Dipartimento di Fisica e Astronomia "Ettore Majorana"

Catania - November 3, 2026

Martina Spoto - ID student number: 1000015868

OACT solar cycles data analysis

Solar Physics final project



Introduction

The Sun's surface is usually covered by dark, cold, highly magnetized spots. They appear, last for days, and some of their properties (like heliographic coordinates, area, number) follow specific patterns over a period of about 11 years: the so-called *solar cycle*. In this report, we plot some sunspot features as a function of time using data from the Catania Solar Astrophysical Observatory (OACT). All the code used for this purpose is avaiable on my GitHub repository:

$$github.com/MarStina2001/OACT_solar_cycles_data_analysis$$

Sunspots

Sunspot have been observed from ground-based observatories since centuries and from satellites since a few decades. They have always been considered a mysterious unknown phenomenon and the more we observe them, the more we understand their working mechanism and their behavior thanks to the multiwavelength electromagnetic observations and the scientific and technological improvement during time. One of the most fascinating facts is that the region above the sunspots are very bright in ultraviolet and X-ray ranges. These regions, called active regions, are marked by a complex and strong magnetic field and often appear as complicated sets of loops at different temperatures, as shown. It is in the active regions that most solar flares[1] occur. Furthermore, some regions, known as coronal holes, are dominated by open magnetic field lines, being the primary source of the high-speed solar wind[2] streams. The coronal holes appear much darker than the quiet Sun, because they are empty of plasma most of the time (see fig.1).

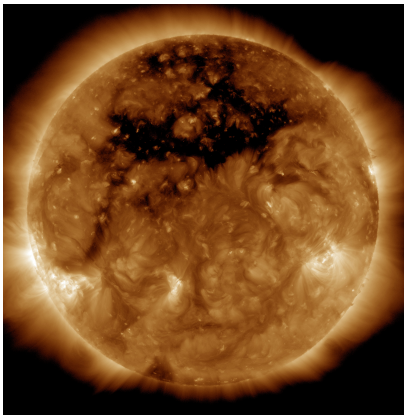


Figure 1: The dark area across the top of the sun in this image is a coronal hole, a region on the sun where the magnetic field is open to interplanetary space, sending coronal material speeding out in what is called a high-speed solar wind stream. The high-speed solar wind originating from this coronal hole, imaged hereon Oct. 10, 2015, by NASA's Solar Dynamics Observatory, created a geomagnetic storm near Earth that resulted in several nights of auroras. This image was taken in wavelengths of 193 Angstroms, which is invisible to our eyes and is typically colorized in bronze. Source: Wikipedia

[1] Flares: huge explosions of electromagnetic radiation from the Sun. They appear as a sudden, intense brightening of a solar region, lasting several minutes to hours, and occur when intense magnetic fields on the Sun become very tangled.

[2] The solar wind is a fast-moving stream of charged particles (plasma) that emerges from the Sun. It travels around the solar system, carrying with it the solar magnetic field. It is a major factor affecting the space environment around the Earth and other planets. On Earth it causes phenomena such as auroras, geomagnetic storms, and radiation belts. It consists mostly of electrons ($\approx 47\%$), protons ($\approx 47\%$), and alpha particles ($\approx 5\%$). It also includes tiny amounts of heavy ions elements like carbon, nitrogen, oxygen, neon, magnesium, silicon, sulfur, iron.

Looking the sunlight at different wavelengths it is possible to observe distinct Sun layers (Photosphere, Chromosphere, Transition Region, Corona) and sunspots are more visible in the photosphere, using the *Continuum* filter, as shown in fig.2. It is possible to observe that sunspots consist of an internal darker region called umbra, a surrounding brighter region called penumbra. The umbra is cooler ($\approx 4000K$) compared to the average temperature of the Photosphere ($\approx 6000K$). The mean dimension of sunspots is $\approx 10000km$, so they are usually bigger than Earth, but pores also exist: penumbra-less sunspots with a diameter $\approx 1000 - 6000km$. Anyway they can reach $\approx 150000km$. We can observe individual sunspots or groups of sunspots: they last from several days to a few months and then decay. Knowing that sunspots are regions of intense magnetic field (of ≈ 3000 Gauss or more compared to the average solar magnetic field of 1 Gauss), it's also possible to observe them with magnetograms like HMI: the Helioseismic and Magnetic Imager on board Solar Dynamics Observatory NASA satellite. In particular, from magnetic polarity and position observations it's also possible to verify Hale's law, two empirical laws that rule the solar cycles.

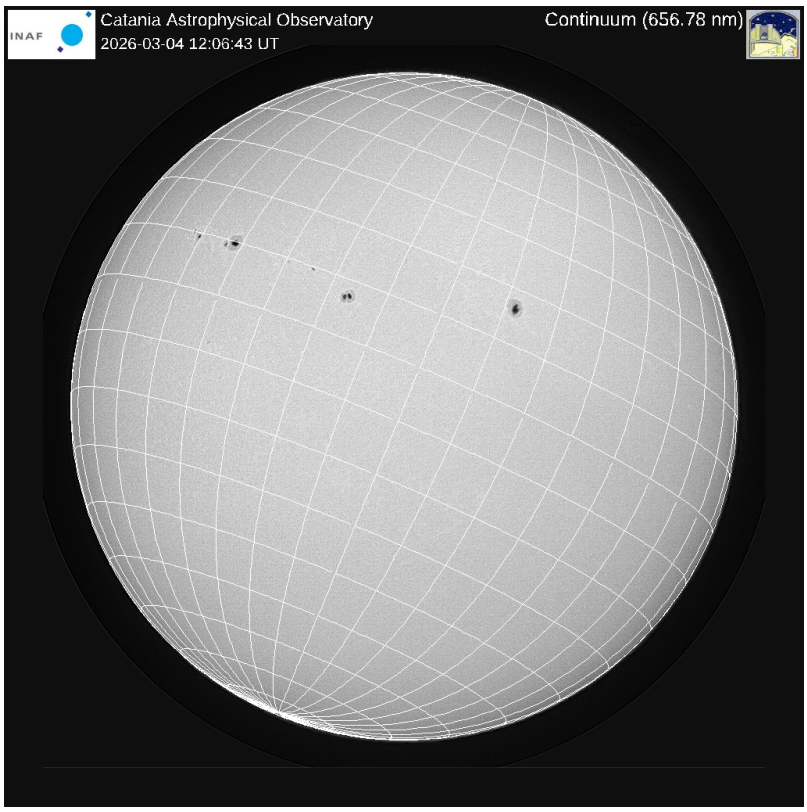


Figure 2: The image shows the continuum image of the Sun acquired at INAF - Catania Astrophysical Observatory on March 4th, 2026. When the weather conditions permit, every 60 minutes (between 8:00 and 14:00 local time - CET/CEST) a full disc image of the Sun in the continuum ($656.78nm \pm 0.25nm$) near the $H\alpha$ line is taken. Although it is obtained by means of a Lyot filter, it provides a view of the solar photosphere as the white light images acquired by other providers/instruments. The raw images are recorded with a size of 2048×2048 pixels and a dynamic range of 16 bit. Then, the image has been preliminarily reduced in order to check its quality and to center the solar disc. Flat-field correction has been applied. Source: OACT Continuum archive

Solar cycle and butterfly diagram

As previously mentioned, some sunspot features change periodically with time. In particular, the sunspot number and the total dark sun surface area follow a nearly-sinusoidal trend with a period of about 11 years. As shown in the average daily sunspot area evolution plot (fig.3), every cycle is numbered, we are in the 25th solar cycle and the highest amplitude variation recorded is in the 19th cycle. From fig.3 it's also possible to note that an interesting *butterfly diagram* results when the sunspot latitude positions are plotted. White light photoheliograms taken over a few days are sufficient to indicate that sunspots are not uniformly distributed over the Sun, but they occur in certain patterns in two latitude zones, called activity zones. These zones are located nearly symmetrically around the solar equator. With the progress of the solar cycle, a migration of the heliographic latitude of sunspot zones towards lower latitudes is observed. This migration is known as *Spörer's law*. Shortly before the sunspot minimum phase, sunspot groups corresponding to the next cycle appear at high latitudes $\pm(30-40°)$. As the cycle progresses, these sunspot groups appear closer towards the equator until just after the next minimum when they disappear in its vicinity.

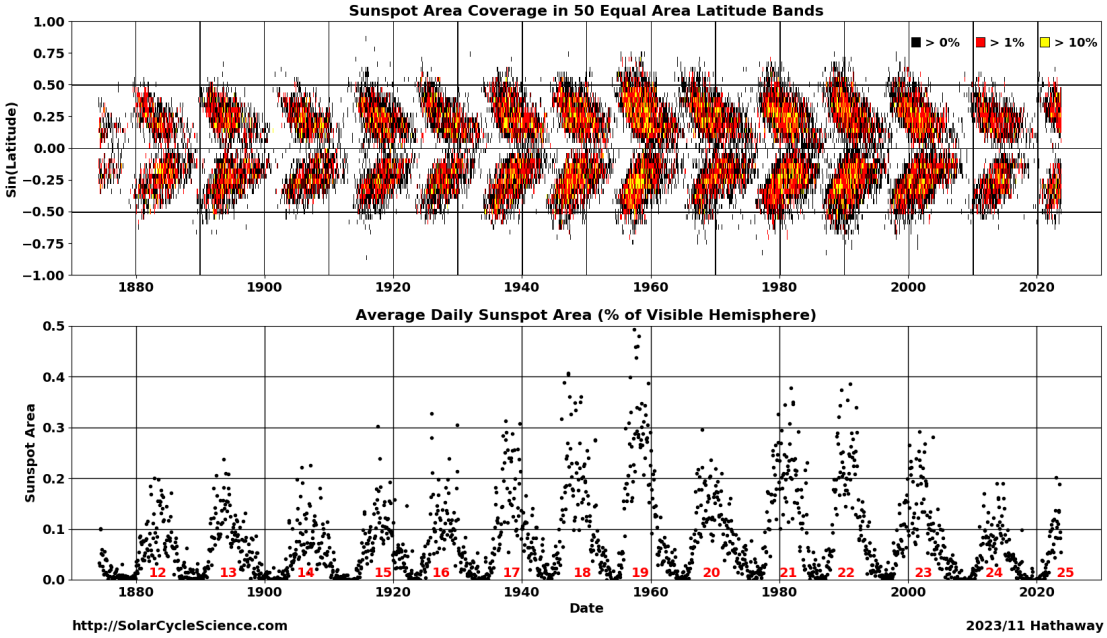


Figure 3: Butterfly diagram and Sunspot area evolution. This version was constructed by the solar group at NASA Marshall Space Flight Center. Source: Wikipedia

We now want to create solar cycle plots like the ones of fig.3 with the data recorded by the Astronomical Observatory of Catania (OACT) in the last 17 years. We will discuss in next sections the methodology and the results.

Methodology

The first step for creating our solar cycles plots is to collect the sunspot features daily recorded by OACT. We can detect sunspots from Continuum digital images in *fits* or *jpg* format by setting a threshold for solar surface pixel darkness, beyond which a group of pixels is identified as a sunspot. But this automatized method would not have any human check. Furthermore, the Continuum image projected on white paper and seen from human eye instead of being recorded on a digital pixel grid has a better resolution. This allows also the observer draw on paper a finer umbra and penumbra sunspot contour. During every working day (from Monday to Friday), the Astrophysical Observatory of Catania collects on its archive[3] the handmade sunspots drawing and store the achieved information in a *txt* file in form of alphanumerical code (ursigram), named USSPS. The OACT website provide an explanation to decode the ursigram:

USSPS 31405 JJHHH aaabc ddeee QXXYY fghii

where:

- JJ = Catania date of observation
- HHH = hours and tenths of hours (UT)
- aaa = Wolf number
- b = seeing quality (1=very poor, 5=exceptional)
- c = solar disk dimension (2 for Catania drawing)
- dd = Catania serial number of sunspot group
- eee = area of sunspot group in tens of millionths of the solar hemisphere
- Q = quadrant (heliographic coordinate) in which geometric center of sunspot group is located:
  - ↗ 1 = NE
  - ↘ 2 = SE
  - ↙ 3 = SW
  - ↖ 4 = NW
- XX = distance to central meridian in degrees
- YY = heliographic latitude in degrees
- f = type of penumbra
- g = relative importance of leading spot and density of the sunspot population
- h = Zurich sunspot class
- ii = number of spots in the group

[3]The archive is available on *https* : *//solar.oact.inaf.it/USSPS_archive.php*

The first line of every USSPS file has the structure you can read on the black box above, the following ones contain only the last 3 strings[4] (the ones that aren't colored in gray). Every USSPS file ends with the line *INAF-CATANIA ASTROPHYSICAL OBSERVATORY*. As you can understand from the explanation given above, every line contains the information of a single sunspots group. For downloading quickly 17 years of daily data we can use the USSPS downloading python code you can read in Appendices. You can customize the code typing the DATE_BEGIN and DATE_END of the time window you want to download the data from and the folder destination of your data DOWNLOAD_DIR. The date string variables are converted in DATETIME OBJECTS by the Python library DATETIME and used for building the URL:

$$http://solar.oact.inaf.it/data/ussps/\{yyyy\}/\{mm\}/\{file_name\}$$

Every single USSPS file URL on the archive has exactly the same shape, so the code just download the files found for every URL that exists using the Python libraries OS and REQUEST[5].
According to the fact that USSPS files are groups of alphanumerical strings, it's easy to separate them with the simplest C standard library <STDIO.H> commands as shown in the appendix code Latitude extraction C code. In particular, the command FSCANF (PF,"%S", S) reads a single string from the txt file and this makes much easier the sunspot features extraction from the USSPS files. Every sunspot feature has been extracted using a different C code: LAT.C extracts sunspot latitude (denoted with Q and YY in ursigram), SUNSPOTSAREA.C extracts sunspots area (eee) and SPOTCOUNT.C extracts sunspots number (ii). Every C code has basically the same structure:

- They read all the USSPS files stored on the DOWNLOAD_DIR folder (be sure that there is nothing else inside) using the <DIRENT.H> C library.
- They skip the first 4 strings of every USSPS file (the ones colored in gray in the ursigram) with the FSCANF command.
- In the for loop FOR (J=0;J<3;J++) we select which of the following 3 strings of the line is the one that contains the feature we are interested in, we convert the chars of that string in integers (q, y1, y2 in the case of LAT.C code).
- We store the information in a destination txt file[6] with the FPRINTF command. In this way we obtain a txt file with the dates in the form DDMMYY.TXT written in every line followed by the sunspot feature in the underlying line.
- Knowing that every USSPS file ends with the line *INAF-CATANIA ASTROPHYSICAL OBSERVATORY*, the for loops are arrested by the command IF (STRCMP (S,"INAF - CATANIA") ==0) BREAK; that belong to the library <STRING.H>

The destination txt files produced by the C codes are read by the Python codes that create the Solar Cycle plots in PNG format, like the appendix code Butterfly diagram maker Python code. All the Python codes have also a similar structure, and they all are into the same Jupyter notebook on my GitHub (OACT_Sun.ipynb). The Butterfly diagram Python code consists of a function (PARSE_AND_PLOT_BUTTERFLY (FILENAME_OR_TEXT)) that creates and fill the abscissae and ordinates arrays of the plot (time and latitude), makes the conversion of arrays in Panda's library objects with the command PD.DATAFRAME and creates and customize the scatter[7] plot. The main function PARSE_AND_PLOT_BUTTERFLY (FILENAME_OR_TEXT) open the destination txt file generated by the C code with the Python command TRY, search the dates considering that every

date is followed by the chars ".TXT", converts the dates in datetime library objects, and write them as a fractional number for a better plotting. Then latitudes are also searched in a similary way to dates with the RE library command.
All the other Python codes for building the other plots work in the same way, with the difference that for a better visualization a curve of a mobile average of 30 days observations has been overplotted and created with the command DF.ROLLING avaiable on the Python pandas library.
The final Sunspots number plot you can find in the Jupyter notebook OACT_Sun.ipynb is not made with OACT data but with SILSO's one, avaiable on its website (sidc.be). Here I downloaded a file containing all the Sunspots number detected on the last 2 centuries on ground observatories in a single txt file. I separated the information with a C code in a way similar to the one of OACT's data and plotted it with Python.

[4] A string is group of chars that are not separated by a blank space.

[5] On my GitHub the thousands output lines of the Python Jupyter cell have been deleted for a better reading of the notebook.

[6] The destination txt file for every C code are LAT.TXT, SUNSPOTSAREA.TXT, SUNSPOTSCOUNT.TXT.

[7] In a scatter Python plots the dots of a given set coordinates (latitude, time) aren't connected by lines.

Results

The final plots in PNG format are the following ones:

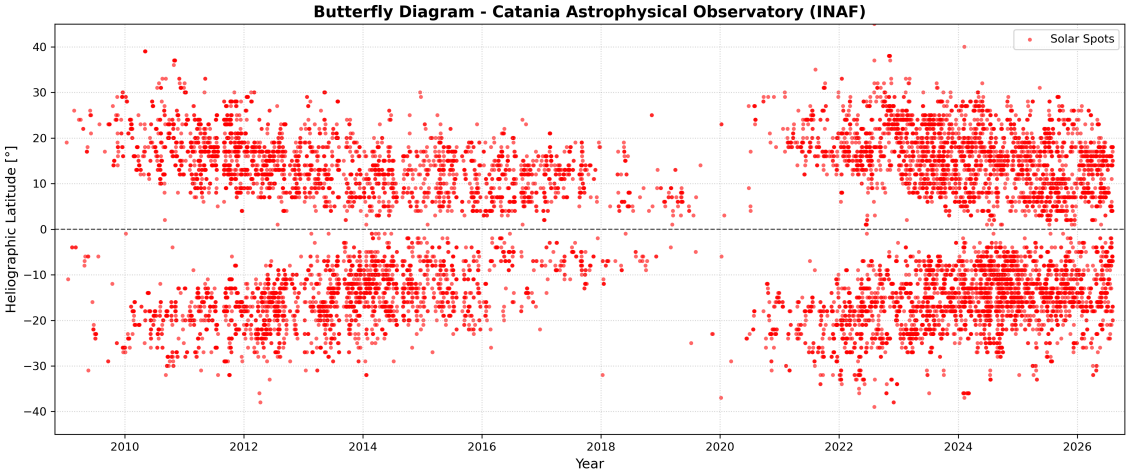


Figure 4: Butterfly diagram of 17 years OACT data obtained by the USSPS files edited by LAT.C and OACT_SUN.IPYNB

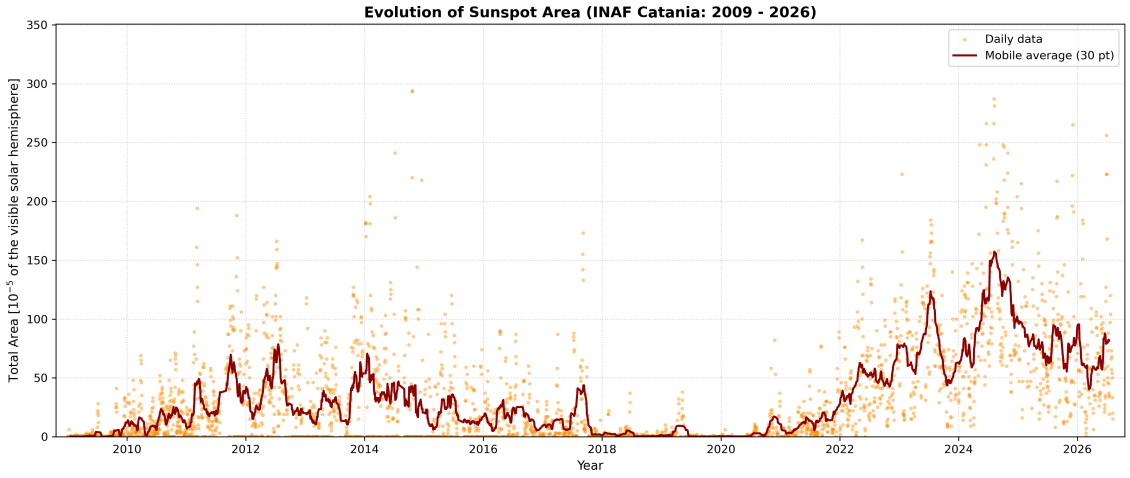


Figure 5: Evolution of sunspots area of 17 years OACT data obtained by the USSPS files edited by SUNSPOTSAREA.C and OACT_SUN.IPYNB

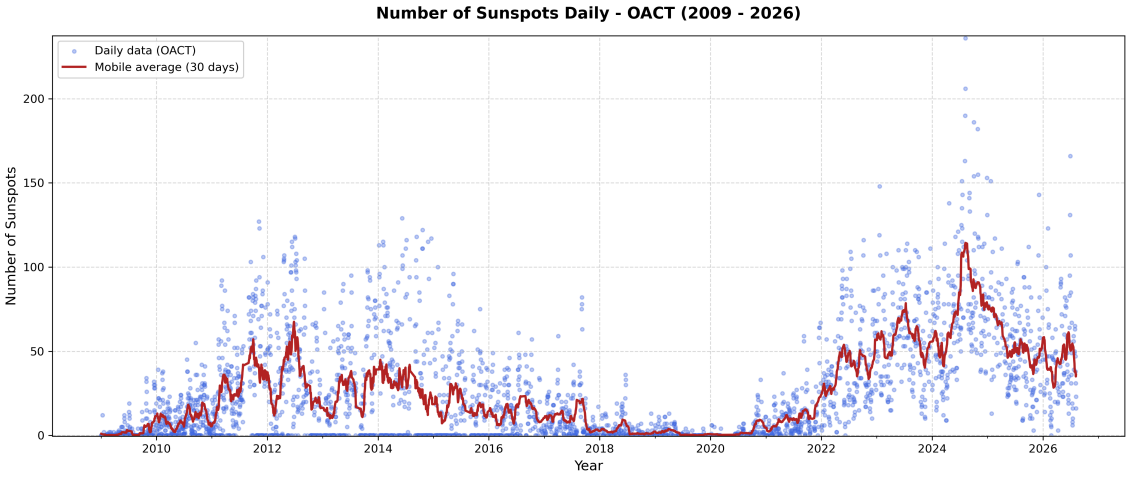


Figure 6: Evolution of the daily sunspots number of 17 years OACT data obtained by the USSPS files edited by SPOTCOUNT.C and OACT_SUN.IPYNB

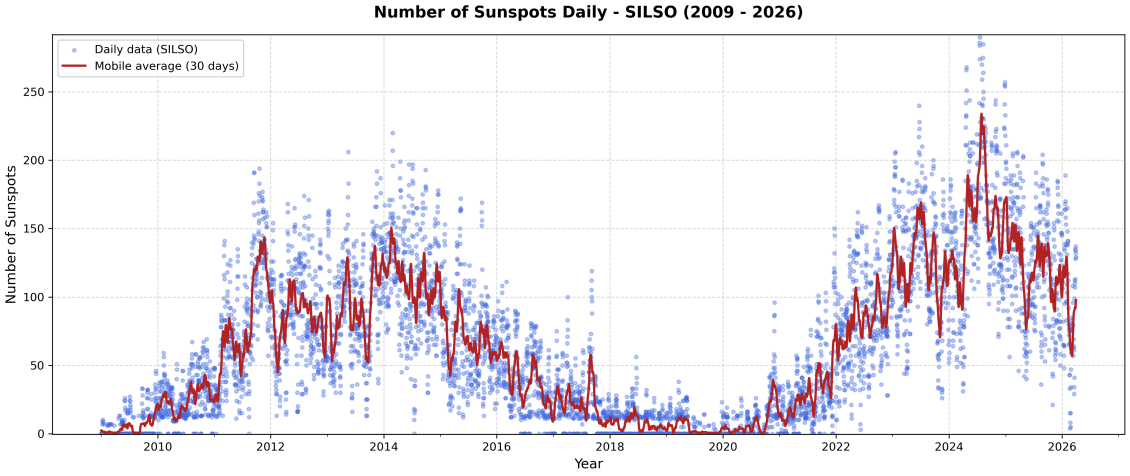


Figure 7: Evolution of the daily sunspots number of 17 years SILSO data obtained by the USSPS files edited by SILSO.C and OACT_SUN.IPYNB

As we expected, the butterfly diagram (fig.4) seems to have a butterfly shape really, as predicted by the Spörer's law and by the NASA (fig.3) observations. The OACT butterfly plot seems to be more noisy than the ones of fig.3 but it depends on the fact that we are looking in a time interval of 17 years and not 2 centuries. It's also intresting to notice that the BUTTERFLY WINGS have the same height (between 30° and 40°) in both Cycle 24 and Cycle 25, differently from the previous cycle that seems to have an upper and lower bigger latitude limit value in both emispheres. The equator line, as expected, is completly free of red dots cause of the fact sunspots never seems to appear at 0° latitude. They are also very visible the 2009 and 2020 minimums and the *quick* change of trend of the sunspots latitude (in about one and a half year around 2020).
The sunspots area evolution plot (fig.5) has been obtained by summing the area of all sunspots groups for every daily USSPS txt file. It's possible to notice that the mobile 30 days average overplotted with the red line on the yellow dots shows approxximatively the same solar cycle trend found in fig.3.
The last two plots obtained by the OACT_SUN.IPYNB are the sunspots number evolution of OACT and SILSO (fig.6 and fig.7). In both cases the mobile 30 days average overplotted with a red line on the blue dots shows the solar cycles trend, and it's clearly visible the 2020 minimum. The main differences between the two plots are: the two solar cycles peaks positions (around 150 and 200 for OACT, around 200 and 250 for SILSO) and the bigger amount of SILSO data (blue dots number) compared with the OACT ones. Instead, the two red lines in both plots seems to have approximatively the same pattern.

Discussion and Conclusions

The OACT Solar Cycles plots obtained by the 17 years archive of the USSPS txt files agree satisfyingly with the Solar Physics literature: the latitude-time plot shows the typical butterfly shape predicted by Spörer's law and the sunspots area and count evolution is modulated by an 11-year cycle.
Looking qualitatively at the OACT butterfly diagram (fig.4), it's possible to notice that the red dots density (sunspots number for given latitude and time) seems bigger in the Cycle 25 rather than in the 24 one. One could think that the OACT has started to be opened for more days during this cycle, and so we can consider that:

$$\frac{11}{17} \approx 0.65 \rightarrow \text{Cycle 24 time bin cover the 65\% of our dataset}$$

But the number of USSPP txt files stored from 01-01-2009 to 01-01-2020 is:

$$\frac{N_{Cycle24}}{N_{tot}} = \frac{2413}{3524} \approx 0.69 \rightarrow 69\% \text{ of our dataset belong to the old cycle}$$

This means that the amount of data recorded in the last cycle is a few percent higher than an expected equally distributed USSPS txt file number density over time. This can't explain the lower red dots density of the Cycle 24. So one could imagine that there has been an improvement of seeing quality in the last 6 years due to better weather condition or instrumental improvement. But, as shown in fig.8, the seeing has been pretty constant during the whole last two observed cycles. Then the reason of this red dots density change in the two cycles seems unknown, a more accurate data set in wich the heliographic latitude of every sunspot instead of every sunspots group is recorded would maybe give different results.

As previously mentioned, the cycles peaks of the sunspots number evolution plot of OACT and SILSO (fig.6 and fig.7) are different, the SILSO ones are sistematically higher and the blue dots density (amount of data) is also bigger. These differences exist due to the fact that SILSO collects all the world sunspots data and so the amount of daily sunspots number has to be bigger for sure. The height peaks difference could maybe depend on the fact that the average over all the world data is mostly not affected by single observatories weather condition (so the total seeing is better). Finally, the codes used for all the plots making could be improved in the future. The whole dataset consists of 3524 txt files, the total weight is less than 2 megabytes. Usually C codes are faster in editing huge amounts of data (Gigabytes), but they don't have parallel computing algorithms like the ones of PyTorch Pithon library or IDL. In our situation, the use of C codes doesn't probably decrease so much the computational time of code executing. So the whole work could have been done in Python only, or I could write a more optimized C background code for building a Python library that extracts from USSPS files all the features that a user can ask.
Furthermore, I could use the USSPS files data set for building a machine learning model that predicts solar flares, training it with the data stored in the website SOLARMONITOR.ORG.

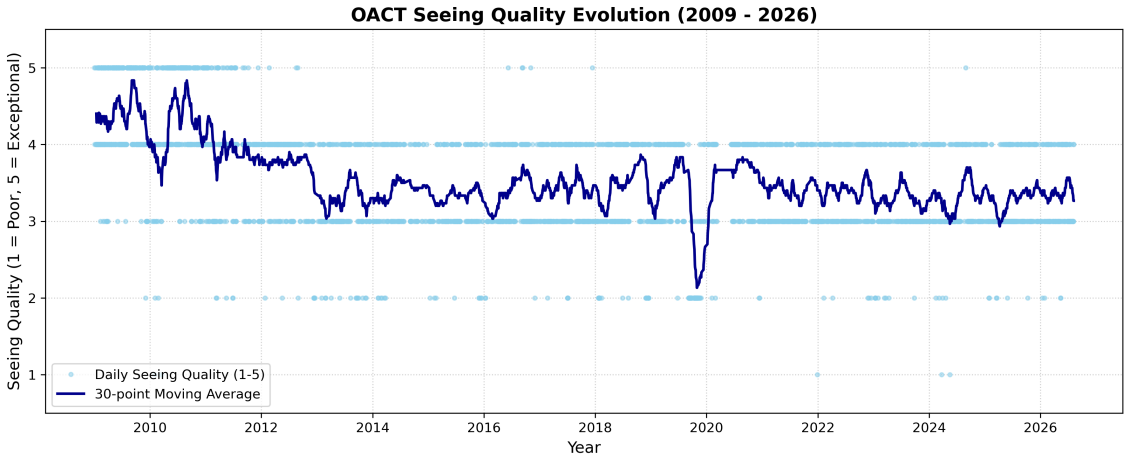


Figure 8: Evolution of the OACT seeing index in the last 17 years.

References

[1] H. M. Antia, A. Bhatnagar, and P. Ulmschneider. *Lectures on Solar Physics*. Vol. 619. Lecture Notes in Physics. Berlin, Heidelberg: Springer-Verlag, 2003. ISBN: 978-3-540-36527-3.

[2] A. Giunta. "Lecture Notes in Solar Physics". 2026.

[3] Egidio Landi Degl'Innocenti. *Fisica Solare*. Milano: Springer-Verlag, 2008. ISBN: 978-88-470-0797-0.

Appendices

USSPS downloading python code

```python
from datetime import datetime, timedelta
import os
import requests

# 1. Set the desired date range here (format: YYYY-MM-DD)
DATE_BEGIN = "2009-01-01"
DATE_END = "2026-08-13"

# Destination directory for downloaded files
DOWNLOAD_DIR = "dati_ussps_txt"
os.makedirs(DOWNLOAD_DIR, exist_ok=True)

# Convert to datetime objects
start_dt = datetime.strptime(DATE_BEGIN, "%Y-%m-%d")
end_dt = datetime.strptime(DATE_END, "%Y-%m-%d")

current_dt = start_dt
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"
}

print(f"--- Download USSPS data from {DATA_BEGIN} to {DATA_END} ---\n")

downloaded = 0
not_found = 0

while current_dt <= end_dt:
    # Extract year, month, and day components
    yyyy = current_dt.strftime("%Y")  # ex. "2026"
    yy = current_dt.strftime("%y")  # ex. "26"
    mm = current_dt.strftime("%m")  # ex. "08"
    dd = current_dt.strftime("%d")  # ex. "03"

    # File name: YYMMDD.txt (ex. 260803.txt)
    file_name = f"{yy}{mm}{dd}.txt"

    # Direct URL to the file on the INAF server
    url = f"http://solar.oact.inaf.it/data/ussps/{yyyy}/{mm}/{file_name}"

    save_path = os.path.join(DOWNLOAD_DIR, file_name)

    try:
        response = requests.get(url, headers=headers)

        if response.status_code == 200:
            with open(save_path, "wb") as f:
                f.write(response.content)
            print(f"[OK] Downloaded: {file_name} -> {url}")
            downloaded += 1
        else:
            print(
                f"[NO] {file_name} not found on the server (Status: {response.
```

```
    status_code})"
            )
            not_found += 1

    except Exception as e:
        print(f"[ERROR] Unable to download {file_name}: {e}")

    # Go to the next day
    current_dt += timedelta(days=1)

print(
    f"\nCompleted! File downloaded: {downloaded} | File not found (e.g., missing observations): {not_found}"
)
print(f"The files are located in the folder: {os.path.abspath(DOWNLOAD_DIR)}")
```

Latitude extraction C code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>

#define DIRNAME "dati_ussps_txt"
#define LAT     "lat.txt"
#define STRLEN  1024

typedef FILE*     pfile;

int main(void)
 {
       int    i,j,q,y1,y2;
       pfile  pf,pflat;
       char   s[STRLEN],filename[STRLEN];
       DIR *d;
       struct dirent *dir;


       if ((pflat=fopen(LAT,"w"))==NULL) goto file_err;

       d = opendir(DIRNAME);  //* This folder is "." *//
       if (d)
       {
         while ((dir = readdir(d)) != NULL) {

           if (strcmp(dir->d_name, ".") == 0 || strcmp(dir->d_name, "..") == 0)
    {
             continue;
           }

           snprintf(filename, sizeof(filename), "%s/%s", DIRNAME, dir->d_name);



       if ((pf=fopen(filename,"r"))==NULL) goto file_err;

```

```
38          fprintf(pflat, "%s\n", filename+15); /*+15 avoids the DIRNAME*/
39
40          for(i=0;i<4;i++)      //*this is always file intro*//
41           {
42               fscanf(pf,"%s",s);
43           }
44
45
46          for(i=4;;i++)
47           {
48               for(j=0;j<3;j++)
49                {
50                   if(strcmp(s,"INAF-CATANIA")==0) break;
51                   fscanf(pf,"%s",s);
52                   if(j==1)
53                     {
54                       q=s[0]-'0'; /*string to integer conversion*/
55                       y1=s[3]-'0';
56                       y2=s[4]-'0';
57                       switch(q)
58                        {
59                         case 1: fprintf(pflat,"%d%dNE\n",y1,y2); break;
60                         case 2: fprintf(pflat,"%d%dSE\n",y1,y2); break;
61                         case 3: fprintf(pflat,"%d%dSW\n",y1,y2); break;
62                         case 4: fprintf(pflat,"%d%dNW\n",y1,y2); break;
63                         default: //printf("ERRORE\n");  /*it would print in spot-
    less days also*/
64                        }
65
66                     }
67                }
68               if(strcmp(s,"INAF-CATANIA")==0) break;
69           }
70
71
72          fclose(pf);
73                        }
74             closedir(d);
75          }
76
77          fclose(pflat);
78
79          return EXIT_SUCCESS;
80
81          file_err: printf("Problems with the file!\n"); return EXIT_FAILURE;
82      }
83
84
85  /*
86
87  To compile this file on terminal:     gcc lat.c -o lat
88  and to execute:                       ./lat
89  Remember you must have the folder 'dati_ussps_txt' with USSPS txt into only
90
91  */
```

Butterfly diagram maker Python code

```
    from datetime import datetime
import re
import matplotlib.pyplot as plt
import pandas as pd


def parse_and_plot_butterfly(filename_or_text):
    dates = []
    latitudes = []

    current_date = None

    # If a file path is provided, read it; otherwise, use the string directly
    try:
        with open(filename_or_text, "r") as f:
            lines = f.readlines()
    except (FileNotFoundError, OSError):
        lines = filename_or_text.strip().split("\n")

    for line in lines:
        line = line.strip()
        if not line:
            continue

        # 1. Recognize the change of file/date (ex: 090110.txt)
        match_file = re.search(r"(\d{6})\.txt", line)
        if match_file:
            yymmdd = match_file.group(1)
            # Convert YYMMDD in datetime object (ex. 09 -> 2009, 26 -> 2026)
            dt = datetime.strptime(yymmdd, "%y%m%d")

            # Convert the date in decimal format (ex. 2009.025)
            # to have a continuous and realistic X axis
            year_start = datetime(dt.year, 1, 1)
            year_end = datetime(dt.year + 1, 1, 1)
            frac_year = dt.year + (dt - year_start) / (year_end - year_start)

            current_date = frac_year
            continue

        # 2. Recognize the latitude values (ex: 19NE, 11SW, 04SE, 24NW)
        match_lat = re.match(r"^(\d{2})(NE|NW|SE|SW)$", line)
        if match_lat and current_date is not None:
            deg = int(match_lat.group(1))
            quadrant = match_lat.group(2)

            # Sign: North (+) / South (-)
            lat = deg if quadrant in ["NE", "NW"] else -deg

            dates.append(current_date)
            latitudes.append(lat)

    # DataFrame creation
    df = pd.DataFrame({"Year": dates, "Latitude": latitudes})

```

```
    # ========================================
    # PLOT: BUTTERFLY DIAGRAM
    # ========================================
    plt.figure(figsize=(14, 6), dpi=150)

    # Scatter plot of data
    plt.scatter(
        df["Year"],
        df["Latitude"],
        c="red",
        s=12,
        alpha=0.6,
        edgecolors="none",
        label="Solar Spots",
    )

    # Sun equator line (Latitude 0 deg)
    plt.axhline(0, color="black", linestyle="--", linewidth=1, alpha=0.7)

    # Axis labels and title
    plt.title(
        "Butterfly Diagram - Catania Astrophysical Observatory (INAF)",
        fontsize=14,
        fontweight="bold",
    )
    plt.xlabel("Year", fontsize=12)
    plt.ylabel("Heliographic Latitude [deg]", fontsize=12)

    # Set axis limits and grid
    plt.ylim(-45, 45)
    plt.xlim(df["Year"].min() - 0.2, df["Year"].max() + 0.2)

    plt.grid(True, linestyle=":", alpha=0.6)
    plt.legend(loc="upper right")
    plt.tight_layout()

    # Show and save the plot
    plt.savefig("butterfly_diagram.png", dpi=300)
    plt.show()


# ========================================
# EXECUTION
# ========================================
if __name__ == "__main__":
    FILE_NAME = "lat.txt"
    parse_and_plot_butterfly(FILE_NAME)
```